

Triggers and Rules in PostgreSQL

Table of Contents

1. [Learning Objectives](#)
 2. [What Are Triggers?](#)
 3. [Trigger Execution Flow — ASCII Diagram](#)
 4. [BEFORE Triggers](#)
 5. [AFTER Triggers](#)
 6. [INSTEAD OF Triggers on Views](#)
 7. [Trigger Functions in PL/pgSQL](#)
 8. [RETURN NEW vs RETURN OLD vs RETURN NULL](#)
 9. [Per-Row vs Per-Statement Triggers](#)
 10. [Trigger Firing Order](#)
 11. [Transition Tables \(Statement-Level NEW/OLD\)](#)
 12. [Rules vs Triggers](#)
 13. [Audit Trigger Pattern](#)
 14. [Row-Versioning Trigger](#)
 15. [Cascading Trigger Problem](#)
 16. [Conditional Triggers — WHEN Clause](#)
 17. [Trigger Introspection Queries](#)
 18. [Common Mistakes](#)
 19. [Best Practices](#)
 20. [Interview Questions & Answers](#)
 21. [Exercises and Solutions](#)
 22. [Cross-References](#)
-

Learning Objectives

By the end of this module you will be able to:

- Distinguish BEFORE, AFTER, and INSTEAD OF triggers and choose the right timing
 - Write trigger functions in PL/pgSQL with correct return values
 - Explain the difference between per-row and per-statement triggers
 - Implement audit logging and row-versioning using triggers
 - Understand INSTEAD OF triggers on views for updatable view patterns
 - Explain why triggers are preferred over rules in modern PostgreSQL
 - Identify and resolve cascading trigger problems
 - Query system catalogs to inspect trigger definitions
-

What Are Triggers?

A **trigger** is a named database object that automatically executes a function in response to a data-change event on a table or view. Triggers allow you to enforce business rules, maintain derived data, and log changes without requiring application-level code.

TRIGGER ANATOMY:

```
CREATE TRIGGER <name>
{ BEFORE | AFTER | INSTEAD OF }
```

```
| { INSERT | UPDATE [ OF col, ... ] | DELETE | TRUNCATE }  
| [ OR { INSERT | UPDATE | DELETE | TRUNCATE } ... ]  
| ON <table_or_view>  
| [ REFERENCING transition_table_clause ]  
| [ FOR EACH { ROW | STATEMENT } ]  
| [ WHEN ( condition ) ]  
| EXECUTE FUNCTION <trigger_function>();
```

KEY CONCEPTS:

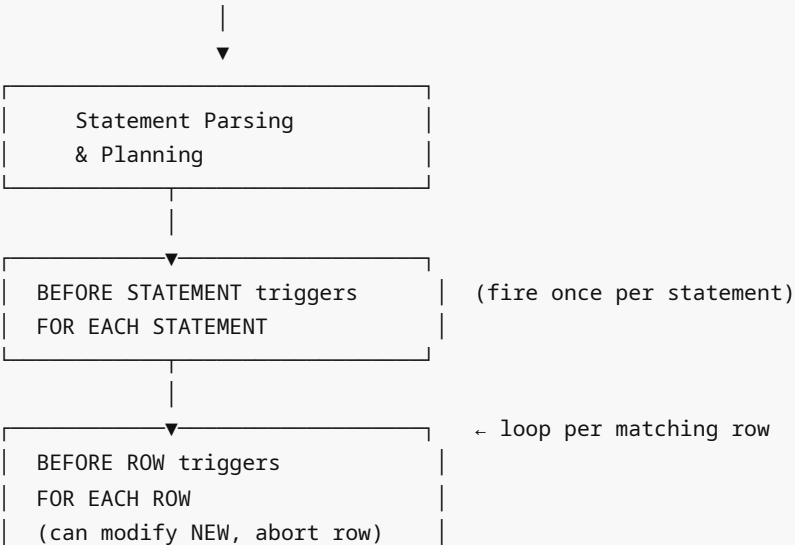
- Timing → BEFORE, AFTER, INSTEAD OF
- Event → INSERT, UPDATE, DELETE, TRUNCATE (can combine with OR)
- Granularity → FOR EACH ROW (fires per modified row)
FOR EACH STATEMENT (fires once per SQL statement)
- Condition → WHEN clause for conditional firing
- Function → A special RETURNS trigger function in PL/pgSQL (or C)

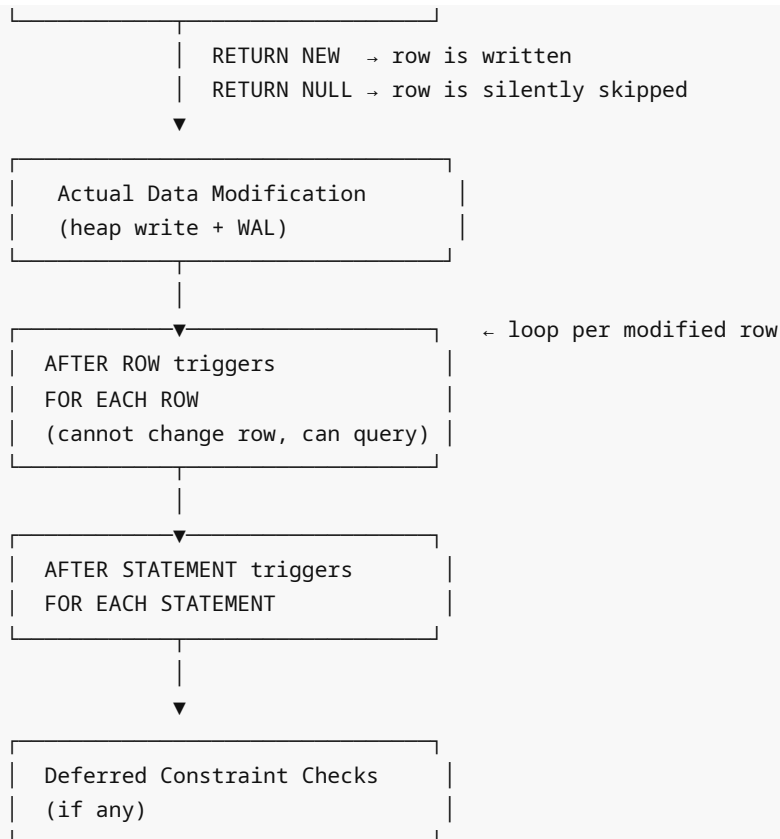
Trigger vs Constraint vs Application Logic

Concern	Trigger	Constraint	App Logic
Data validation	Yes (complex)	Yes (simple)	Yes
Cross-table enforcement	Yes	FK only	Yes
Audit logging	Yes (best)	No	Yes (fragile)
Derived column maintenance	Yes	No	Yes
Performance overhead	Moderate	Low	None (DB)
Bypass risk	Hard (DDL)	Hard	Easy (bug)

Trigger Execution Flow — ASCII Diagram

```
CLIENT SENDS: UPDATE orders SET status = 'shipped' WHERE id = 42;
```





INSTEAD OF (views only):

Replaces the data-modification attempt entirely.

No actual write happens unless the function performs it explicitly.

BEFORE Triggers

BEFORE triggers fire **before the row is written** to the heap. They can:

- Modify `NEW` to alter what gets stored
- `RETURN NULL` to silently cancel the operation for this row
- Raise exceptions to abort the entire statement

Example 1 — Normalize data before INSERT

```
-- Ensure email is always stored lowercase and trimmed
CREATE OR REPLACE FUNCTION trg_normalize_user_email()
RETURNS trigger
LANGUAGE plpgsql AS
$$
BEGIN
    NEW.email := lower(trim(NEW.email));
    NEW.username := lower(trim(NEW.username));
    -- Auto-set created_at if not provided
    IF NEW.created_at IS NULL THEN
        NEW.created_at := now();
    END IF;
END;
```

```

        END IF;
        RETURN NEW; -- RETURN NEW to apply the modified row
    END;
$$;

CREATE TRIGGER normalize_user_email
BEFORE INSERT OR UPDATE OF email, username
ON users
FOR EACH ROW
EXECUTE FUNCTION trg_normalize_user_email();

-- Test
INSERT INTO users (email, username) VALUES (' Alice@Example.COM ', ' Alice ');
-- Stored as: email='alice@example.com', username='alice'

```

Example 2 — Block a forbidden state transition

```

CREATE OR REPLACE FUNCTION trg_enforce_order_state_machine()
RETURNS trigger
LANGUAGE plpgsql AS
$$
DECLARE
    v_valid_transitions TEXT[][] := ARRAY[
        ARRAY['pending', 'confirmed'],
        ARRAY['confirmed', 'shipped'],
        ARRAY['shipped', 'delivered'],
        ARRAY['pending', 'cancelled'],
        ARRAY['confirmed', 'cancelled']
    ];
    v_pair TEXT[];
    v_ok BOOLEAN := FALSE;
BEGIN
    IF NEW.status = OLD.status THEN
        RETURN NEW; -- no change, allow
    END IF;

    FOREACH v_pair SLICE 1 IN ARRAY v_valid_transitions LOOP
        IF v_pair[1] = OLD.status AND v_pair[2] = NEW.status THEN
            v_ok := TRUE;
            EXIT;
        END IF;
    END LOOP;

    IF NOT v_ok THEN
        RAISE EXCEPTION 'Invalid order state transition: % → %',
            OLD.status, NEW.status
            USING ERRCODE = 'check_violation';
    END IF;

    RETURN NEW;
END;

```

```

$$;

CREATE TRIGGER enforce_order_state_machine
BEFORE UPDATE OF status
ON orders
FOR EACH ROW
EXECUTE FUNCTION trg_enforce_order_state_machine();

```

Example 3 — Computed column (before insert/update)

```

-- Maintain a full_name column automatically
CREATE OR REPLACE FUNCTION trg_compute_full_name()
RETURNS trigger
LANGUAGE plpgsql AS
$$
BEGIN
    NEW.full_name := trim(coalesce(NEW.first_name, '') || ' ' ||
coalesce(NEW.last_name, ''));
    RETURN NEW;
END;
$$;

CREATE TRIGGER compute_full_name
BEFORE INSERT OR UPDATE OF first_name, last_name
ON employees
FOR EACH ROW
EXECUTE FUNCTION trg_compute_full_name();

```

AFTER Triggers

AFTER triggers fire **after the row has been written** to the heap. They:

- Cannot modify the row that was just written (RETURN value is ignored for row)
- Can query the newly written data
- Can INSERT/UPDATE/DELETE other tables
- Are commonly used for audit logs, notifications, and maintaining denormalized data

Example 4 — Send notification after INSERT

```

CREATE OR REPLACE FUNCTION trg_notify_new_order()
RETURNS trigger
LANGUAGE plpgsql AS
$$
BEGIN
    -- pg_notify sends a lightweight asynchronous notification
    PERFORM pg_notify(
        'new_order',
        json_build_object(
            'order_id', NEW.id,
            'customer_id', NEW.customer_id,

```

```

        'total', NEW.total_amount
    )::text
);
RETURN NEW; -- AFTER ROW: return value is ignored, but convention is RETURN NEW
END;
$$;

CREATE TRIGGER notify_new_order
AFTER INSERT
ON orders
FOR EACH ROW
EXECUTE FUNCTION trg_notify_new_order();

```

Example 5 — Maintain a summary/denormalized table

```

-- Keep a running order count per customer
CREATE TABLE customer_stats (
    customer_id BIGINT PRIMARY KEY,
    order_count INT NOT NULL DEFAULT 0,
    total_spent NUMERIC NOT NULL DEFAULT 0,
    last_order TIMESTAMPTZ
);

CREATE OR REPLACE FUNCTION trg_update_customer_stats()
RETURNS trigger
LANGUAGE plpgsql AS
$$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO customer_stats (customer_id, order_count, total_spent,
last_order)
VALUES (NEW.customer_id, 1, NEW.total_amount, NEW.created_at)
ON CONFLICT (customer_id) DO UPDATE SET
    order_count = customer_stats.order_count + 1,
    total_spent = customer_stats.total_spent + EXCLUDED.total_spent,
    last_order = GREATEST(customer_stats.last_order, EXCLUDED.last_order);

    ELSIF TG_OP = 'DELETE' THEN
        UPDATE customer_stats
        SET     order_count = order_count - 1,
              total_spent = total_spent - OLD.total_amount
        WHERE  customer_id = OLD.customer_id;
    END IF;

    RETURN NULL; -- AFTER ROW: return value ignored; RETURN NULL is fine
END;
$$;

CREATE TRIGGER update_customer_stats
AFTER INSERT OR DELETE
ON orders

```

```
FOR EACH ROW
EXECUTE FUNCTION trg_update_customer_stats();
```

INSTEAD OF Triggers on Views

INSTEAD OF triggers apply **only to views**. They replace the default behavior (which would be an error for non-updatable views) with custom logic.

Example 6 — Updatable view via INSTEAD OF trigger

```
-- View joins two tables; normally not directly updatable
CREATE VIEW employee_with_dept AS
SELECT
    e.id,
    e.full_name,
    e.salary,
    d.id AS dept_id,
    d.name AS dept_name
FROM employees e
JOIN departments d ON d.id = e.dept_id;

-- Make it updatable for salary changes
CREATE OR REPLACE FUNCTION trg_update_employee_view()
RETURNS trigger
LANGUAGE plpgsql AS
$$
BEGIN
    IF TG_OP = 'UPDATE' THEN
        UPDATE employees
        SET salary = NEW.salary,
            full_name = NEW.full_name
        WHERE id = NEW.id;
        RETURN NEW;
    ELSIF TG_OP = 'INSERT' THEN
        INSERT INTO employees (full_name, salary, dept_id)
        VALUES (NEW.full_name, NEW.salary, NEW.dept_id)
        RETURNING id INTO NEW.id;
        RETURN NEW;
    ELSIF TG_OP = 'DELETE' THEN
        DELETE FROM employees WHERE id = OLD.id;
        RETURN OLD;
    END IF;
END;
$$;

CREATE TRIGGER employee_view_dml
INSTEAD OF INSERT OR UPDATE OR DELETE
ON employee_with_dept
FOR EACH ROW
EXECUTE FUNCTION trg_update_employee_view();
```

```
-- Now this works:  
UPDATE employee_with_dept SET salary = 95000 WHERE id = 101;
```

Trigger Functions in PL/pgSQL

A trigger function is a **special function** that returns `trigger` (not a regular data type).

Special Variables Available Inside Trigger Functions

```
-- TG_NAME      - name of the trigger that fired  
-- TG_WHEN      - 'BEFORE', 'AFTER', or 'INSTEAD OF'  
-- TG_LEVEL     - 'ROW' or 'STATEMENT'  
-- TG_OP        - 'INSERT', 'UPDATE', 'DELETE', or 'TRUNCATE'  
-- TG_TABLE_NAME - name of the table  
-- TG_TABLE_SCHEMA - schema of the table  
-- TG_NARGS     - number of arguments passed to the function  
-- TG_ARGV[]    - array of text arguments (0-indexed)  
-- NEW          - new row for INSERT/UPDATE (RECORD type; NULL for DELETE)  
-- OLD          - old row for UPDATE/DELETE (RECORD type; NULL for INSERT)  
  
CREATE OR REPLACE FUNCTION generic_debug_trigger()  
RETURNS trigger  
LANGUAGE plpgsql AS  
$$  
BEGIN  
    RAISE NOTICE '[TRIGGER] % % % on %.% (row level: %)',  
        TG_NAME, TG_WHEN, TG_OP,  
        TG_TABLE_SCHEMA, TG_TABLE_NAME, TG_LEVEL;  
  
    IF TG_OP = 'INSERT' THEN  
        RAISE NOTICE ' NEW = %', row_to_json(NEW);  
    ELSIF TG_OP = 'UPDATE' THEN  
        RAISE NOTICE ' OLD = %', row_to_json(OLD);  
        RAISE NOTICE ' NEW = %', row_to_json(NEW);  
    ELSIF TG_OP = 'DELETE' THEN  
        RAISE NOTICE ' OLD = %', row_to_json(OLD);  
    END IF;  
  
    RETURN COALESCE(NEW, OLD);  
END;  
$;
```

Passing Arguments to Trigger Functions

```
-- Same function, different behavior per table via TG_ARGV  
CREATE OR REPLACE FUNCTION trg_set_updated_at()  
RETURNS trigger  
LANGUAGE plpgsql AS
```



```

$$
DECLARE
    v_column TEXT := COALESCE(TG_ARGV[0], 'updated_at');
BEGIN
    -- Dynamically set the timestamp column (passed as argument)
    NEW := NEW #= hstore(v_column, now()::text);
    RETURN NEW;
END;
$$;

-- Use with different column names on different tables
CREATE TRIGGER set_updated_at
BEFORE UPDATE ON orders
FOR EACH ROW EXECUTE FUNCTION trg_set_updated_at('updated_at');

CREATE TRIGGER set_modified_at
BEFORE UPDATE ON products
FOR EACH ROW EXECUTE FUNCTION trg_set_updated_at('modified_at');

```

RETURN NEW vs RETURN OLD vs RETURN NULL

This is **one of the most frequently confused areas** in PostgreSQL triggers.

Return	Effect
RETURN NEW	BEFORE INSERT/UPDATE: the returned record is what gets written to the table. Modify NEW before returning to alter what is stored. AFTER ROW: return value is ignored (use RETURN NEW by convention).
RETURN OLD	BEFORE UPDATE: writes OLD (the unmodified row). Rarely useful – effectively cancels the UPDATE's changes while still counting it as a write. BEFORE DELETE: ignored (row is still deleted unless RETURN NULL).
RETURN NULL	BEFORE ROW: silently suppresses this row's operation. The INSERT/UPDATE/DELETE does NOT happen for this row. AFTER ROW / STATEMENT: return value is ignored.
RAISE EXCEPTION	Aborts the entire statement (and transaction unless caught). Works from any trigger timing.

Quick rules:

```

BEFORE INSERT → RETURN NEW (or modified NEW)
BEFORE UPDATE → RETURN NEW (or modified NEW)

```

```
BEFORE DELETE → RETURN OLD (return value ignored; RETURN NULL cancels)
AFTER *      → RETURN NEW (or RETURN NULL – either works)
```

Example 7 — Using RETURN NULL to conditionally suppress inserts

```
-- Discard duplicate events silently (idempotency guard)
CREATE OR REPLACE FUNCTION trg_deduplicate_events()
RETURNS trigger
LANGUAGE plpgsql AS
$$
BEGIN
    -- If an identical event was received in the last 5 seconds, drop it
    IF EXISTS (
        SELECT 1 FROM events
        WHERE event_type = NEW.event_type
        AND     source_id = NEW.source_id
        AND     created_at > now() - interval '5 seconds'
    ) THEN
        RETURN NULL; -- silently suppress this INSERT
    END IF;

    RETURN NEW; -- allow the INSERT
END;
$$;

CREATE TRIGGER deduplicate_events
BEFORE INSERT ON events
FOR EACH ROW
EXECUTE FUNCTION trg_deduplicate_events();
```

Per-Row vs Per-Statement Triggers

FOR EACH ROW:

- Fires once per affected row
- Has access to OLD and NEW records
- Can modify NEW (BEFORE only)
- Higher overhead for bulk operations

FOR EACH STATEMENT:

- Fires once per SQL statement, regardless of rows affected
- OLD and NEW are NULL (use transition tables for data access)
- Lower overhead for bulk operations
- Cannot modify individual rows

Example:

```
UPDATE orders SET processed = TRUE WHERE status = 'shipped';
-- Affects 500 rows
```

```
FOR EACH ROW trigger → fires 500 times
FOR EACH STATEMENT → fires 1 time
```

Example 8 — Per-statement trigger for bulk change tracking

```
CREATE OR REPLACE FUNCTION trg_log_bulk_operation()
RETURNS trigger
LANGUAGE plpgsql AS
$$
BEGIN
    INSERT INTO operation_log (table_name, operation, performed_by, performed_at)
    VALUES (TG_TABLE_NAME, TG_OP, current_user, now());
    RETURN NULL;
END;
$$;

CREATE TRIGGER log_bulk_operation
AFTER INSERT OR UPDATE OR DELETE
ON orders
FOR EACH STATEMENT
EXECUTE FUNCTION trg_log_bulk_operation();
```

Trigger Firing Order

When multiple triggers exist on the same table for the same event and timing:

FIRING ORDER RULES:

1. BEFORE triggers fire before AFTER triggers (by definition).
2. Within the same timing (e.g., all BEFORE INSERT):
→ Triggers fire in ALPHABETICAL ORDER by trigger name.
3. DEFERRED constraint triggers fire at end of transaction.

Practical implication:

Name your triggers with prefixes to control order:

```
10_normalize_email    (fires first)
20_validate_domain    (fires second)
30_set_defaults        (fires third)
```

Example 9 — Controlling trigger order with naming

```
-- These fire in alphabetical order: 10_ before 20_ before 30_
CREATE TRIGGER "10_normalize_contact_data"
BEFORE INSERT OR UPDATE ON contacts
FOR EACH ROW EXECUTE FUNCTION trg_normalize_contact_data();

CREATE TRIGGER "20_validate_contact_data"
BEFORE INSERT OR UPDATE ON contacts
FOR EACH ROW EXECUTE FUNCTION trg_validate_contact_data();
```

```
CREATE TRIGGER "30_set_contact_defaults"
BEFORE INSERT ON contacts
FOR EACH ROW EXECUTE FUNCTION trg_set_contact_defaults();
```

Transition Tables (Statement-Level NEW/OLD)

PostgreSQL 10+ supports `REFERENCING` to expose transition tables — a snapshot of all changed rows as a set — for statement-level triggers.

Example 10 — Transition tables for bulk audit

```
CREATE OR REPLACE FUNCTION trg_audit_bulk_price_change()
RETURNS trigger
LANGUAGE plpgsql AS
$$
BEGIN
    -- 'new_prices' and 'old_prices' are the transition table names
    INSERT INTO price_change_audit (product_id, old_price, new_price, changed_at)
    SELECT
        n.id,
        o.price AS old_price,
        n.price AS new_price,
        now()
    FROM new_prices n
    JOIN old_prices o ON o.id = n.id
    WHERE n.price <> o.price;

    RETURN NULL;
END;
$$;

CREATE TRIGGER audit_bulk_price_change
AFTER UPDATE ON products
REFERENCING OLD TABLE AS old_prices NEW TABLE AS new_prices
FOR EACH STATEMENT
EXECUTE FUNCTION trg_audit_bulk_price_change();
```

Rules vs Triggers

PostgreSQL has a **rules system** (query rewriting) that predates triggers. Modern PostgreSQL strongly discourages rules for DML in favor of triggers.

Comparison

Aspect	Rules	Triggers
When applied	Query rewrite phase (before execution)	Execution phase (during data modification)

Visibility	Invisible to application; rewrites the query	Explicit; visible in pg_trigger
RETURN value	N/A	Controls what is written
Multiple events	One rule per event	One trigger for multiple events
Transaction awareness	Can behave unexpectedly	Fully transactional
Debugging	Very hard	Easier (RAISE NOTICE, logs)
INSTEAD OF on views	Views only (historical)	Views only (preferred)
Recommendation	Avoid for DML	Preferred

When Rules Are Still Used

```
-- The main legitimate use case: DO NOTHING rule for an insert-only view
CREATE RULE "_RETURN" AS ON SELECT TO my_view DO INSTEAD
    SELECT * FROM base_table WHERE active = TRUE;

-- DO NOTHING to make a view ignore certain operations
CREATE RULE no_delete ON orders DO INSTEAD NOTHING;
-- (A trigger with RETURN NULL is cleaner but rules still work here)
```

Example 11 — The same behavior: rule vs trigger

```
-- RULE approach (avoid):
CREATE RULE update_view_rule AS ON UPDATE TO employee_view
DO INSTEAD UPDATE employees SET salary = NEW.salary WHERE id = NEW.id;

-- TRIGGER approach (preferred):
CREATE OR REPLACE FUNCTION trg_update_employee_view_salary()
RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
    UPDATE employees SET salary = NEW.salary WHERE id = NEW.id;
    RETURN NEW;
END;
$$;
CREATE TRIGGER update_employee_view_salary
INSTEAD OF UPDATE ON employee_view
FOR EACH ROW EXECUTE FUNCTION trg_update_employee_view_salary();
-- Triggers: easier to debug, easier to reason about, standard across DBs
```

Audit Trigger Pattern

A complete, production-ready audit trigger that logs all DML changes to a generic audit table.

Example 12 — Generic audit trigger

```
-- Audit table
CREATE TABLE audit_log (
    id          BIGSERIAL    PRIMARY KEY,
    table_name  TEXT         NOT NULL,
    operation   TEXT         NOT NULL,    -- INSERT / UPDATE / DELETE
    row_id      TEXT,        -- cast of primary key
    old_data    JSONB,
    new_data    JSONB,
    changed_by  TEXT         NOT NULL DEFAULT current_user,
    changed_at  TIMESTAMPTZ  NOT NULL DEFAULT now(),
    app_user    TEXT,        -- from session variable
    client_ip   INET
);

CREATE INDEX ON audit_log (table_name, changed_at DESC);
CREATE INDEX ON audit_log USING GIN (new_data);

-- Generic trigger function – can be reused across many tables
CREATE OR REPLACE FUNCTION trg_generic_audit()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER    -- runs as the function owner, not caller
SET search_path = public AS
$$
DECLARE
    v_old_data JSONB;
    v_new_data JSONB;
    v_row_id   TEXT;
BEGIN
    IF TG_OP = 'INSERT' THEN
        v_new_data := row_to_json(NEW)::JSONB;
        v_row_id   := NEW.id::TEXT;
    ELSIF TG_OP = 'UPDATE' THEN
        v_old_data := row_to_json(OLD)::JSONB;
        v_new_data := row_to_json(NEW)::JSONB;
        v_row_id   := NEW.id::TEXT;
    ELSIF TG_OP = 'DELETE' THEN
        v_old_data := row_to_json(OLD)::JSONB;
        v_row_id   := OLD.id::TEXT;
    END IF;

    INSERT INTO audit_log (
        table_name, operation, row_id,
        old_data, new_data,
        app_user, client_ip
    )
    VALUES (
        TG_TABLE_NAME,
        TG_OP,
```

```

        v_row_id,
        v_old_data,
        v_new_data,
        current_setting('app.current_user', true),
        inet_client_addr()
    );

    RETURN COALESCE(NEW, OLD);
END;
$$;

-- Attach to any table with one line
CREATE TRIGGER audit_orders
AFTER INSERT OR UPDATE OR DELETE ON orders
FOR EACH ROW EXECUTE FUNCTION trg_generic_audit();

CREATE TRIGGER audit_customers
AFTER INSERT OR UPDATE OR DELETE ON customers
FOR EACH ROW EXECUTE FUNCTION trg_generic_audit();

-- Set application user in your app before DML:
-- SET LOCAL app.current_user = 'john@example.com';

```

Querying the Audit Log

```

-- All changes to order #1042 in the last 7 days
SELECT
    changed_at,
    operation,
    changed_by,
    app_user,
    jsonb_diff_val(old_data, new_data) AS changed_fields
FROM audit_log
WHERE table_name = 'orders'
    AND row_id = '1042'
    AND changed_at > now() - interval '7 days'
ORDER BY changed_at DESC;

-- Find who deleted rows from customers yesterday
SELECT changed_at, changed_by, app_user, old_data
FROM audit_log
WHERE table_name = 'customers'
    AND operation = 'DELETE'
    AND changed_at::date = current_date - 1;

```

Row-Versioning Trigger

Row versioning adds an auto-incrementing `version` counter and records `created_at` / `updated_at` timestamps on every write.

Example 13 — Row versioning trigger

```
-- Ensure these columns exist on the target table:
-- version      BIGINT NOT NULL DEFAULT 1
-- created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
-- updated_at   TIMESTAMPTZ NOT NULL DEFAULT now()

CREATE OR REPLACE FUNCTION trg_row_versioning()
RETURNS trigger
LANGUAGE plpgsql AS
$$
BEGIN
    IF TG_OP = 'INSERT' THEN
        NEW.version      := 1;
        NEW.created_at   := COALESCE(NEW.created_at, now());
        NEW.updated_at   := now();

    ELSIF TG_OP = 'UPDATE' THEN
        -- Prevent clock skew from overriding version on concurrent updates
        IF NEW.version <> OLD.version THEN
            RAISE EXCEPTION
                'Optimistic lock violation on % row %: expected version %, got %',
                TG_TABLE_SCHEMA, TG_TABLE_NAME, OLD.id, OLD.version, NEW.version
            USING ERRCODE = 'serialization_failure';
        END IF;
        NEW.version      := OLD.version + 1;
        NEW.created_at   := OLD.created_at; -- never change created_at
        NEW.updated_at   := now();
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER row_versioning
BEFORE INSERT OR UPDATE ON products
FOR EACH ROW EXECUTE FUNCTION trg_row_versioning();

-- Optimistic locking pattern in application:
-- UPDATE products SET name='New Name', version=<current_version>
-- WHERE id=42 AND version=<current_version>;
-- → If 0 rows updated, another process changed the row first
```

Cascading Trigger Problem

A **cascading trigger** occurs when a trigger's action fires another trigger, which fires another, potentially creating infinite loops or unexpected side effects.

CASCADING SCENARIO:

UPDATE orders → trigger A → UPDATE order_items → trigger B → UPDATE orders


```
      |
      | UPDATE orders (again!)
      |
      | loop...
```

Example 14 — Detecting and breaking trigger loops

```
-- BAD: Two triggers that can loop
-- Trigger on orders → updates summary table
-- Trigger on summary table → updates orders.total (loops back!)

-- SOLUTION 1: Use a session variable guard
CREATE OR REPLACE FUNCTION trg_update_order_total_safe()
RETURNS trigger
LANGUAGE plpgsql AS
$$
BEGIN
    -- If we're already inside this trigger, skip
    IF current_setting('app.in_order_total_trigger', true) = 'true' THEN
        RETURN NEW;
    END IF;

    -- Set guard
    PERFORM set_config('app.in_order_total_trigger', 'true', true); -- true = local
to txn

    -- Do the work
    UPDATE orders
    SET total_amount = (
        SELECT sum(quantity * unit_price)
        FROM   order_items
        WHERE  order_id = NEW.order_id
    )
    WHERE id = NEW.order_id;

    RETURN NEW;
END;
$$;

-- SOLUTION 2: Check if the value actually changed (avoid redundant updates)
CREATE OR REPLACE FUNCTION trg_sync_denormalized_safe()
RETURNS trigger
LANGUAGE plpgsql AS
$$
BEGIN
    -- Only fire downstream update if the relevant column actually changed
    IF NEW.status IS NOT DISTINCT FROM OLD.status THEN
        RETURN NEW; -- no real change, stop the chain
    END IF;

    -- Real work here...
```

```
        RETURN NEW;
END;
$$;
```

Example 15 — Disabling a trigger temporarily

```
-- Disable trigger for a bulk load (requires SUPERUSER or table owner)
ALTER TABLE orders DISABLE TRIGGER audit_orders;

-- Bulk operation
COPY orders FROM '/tmp/orders_bulk.csv' WITH (FORMAT CSV);

-- Re-enable
ALTER TABLE orders ENABLE TRIGGER audit_orders;

-- Disable ALL triggers on a table (includes system triggers like FK checks –
-- dangerous!)
ALTER TABLE orders DISABLE TRIGGER ALL;
-- Use with extreme caution; FK constraints will not be checked
```

Conditional Triggers — WHEN Clause

The `WHEN` clause allows a trigger to fire only when a condition is true, avoiding the overhead of entering the trigger function for every row.

Example 16 — WHEN clause optimization

```
-- Only audit rows where amount > 10,000
CREATE TRIGGER audit_large_transactions
AFTER INSERT OR UPDATE ON transactions
FOR EACH ROW
WHEN (NEW.amount > 10000)
EXECUTE FUNCTION trg_generic_audit();

-- Only fire on actual status change (not a no-op update)
CREATE TRIGGER sync_on_status_change
AFTER UPDATE OF status ON orders
FOR EACH ROW
WHEN (OLD.status IS DISTINCT FROM NEW.status)
EXECUTE FUNCTION trg_sync_order_status_to_shipping_service();

-- WHEN with OLD and NEW comparison
CREATE TRIGGER track_price_increase
AFTER UPDATE ON products
FOR EACH ROW
WHEN (NEW.price > OLD.price)
EXECUTE FUNCTION trg_log_price_increase();
```

Trigger Introspection Queries

```
-- List all triggers on a specific table
SELECT
    t.tgname          AS trigger_name,
    CASE t.tgtype & 2 WHEN 2 THEN 'BEFORE' ELSE 'AFTER' END AS timing,
    CASE
        WHEN t.tgtype & 4 = 4 THEN 'INSERT'
        WHEN t.tgtype & 8 = 8 THEN 'DELETE'
        WHEN t.tgtype & 16 = 16 THEN 'UPDATE'
    END              AS event,
    CASE t.tgtype & 1 WHEN 1 THEN 'ROW' ELSE 'STATEMENT' END AS level,
    p.proname         AS function_name,
    t.tgenabled       AS enabled    -- 'O' = enabled, 'D' = disabled
FROM   pg_trigger t
JOIN   pg_class   c ON c.oid = t.tgrelid
JOIN   pg_proc    p ON p.oid = t.tgfoid
WHERE  c.relname = 'orders'
AND    NOT t.tgisinternal    -- exclude FK constraint triggers
ORDER BY timing, event, trigger_name;

-- All trigger functions and their source
SELECT
    p.proname    AS function_name,
    pg_get_functiondef(p.oid) AS definition
FROM   pg_proc p
JOIN   pg_language l ON l.oid = p.prolang
WHERE  l.lanname = 'plpgsql'
AND    p.prorettype = 'pg_catalog.trigger'::pg_catalog.regtype;

-- How many triggers exist per table in current database
SELECT
    c.relname AS table_name,
    count(*)  AS trigger_count
FROM   pg_trigger t
JOIN   pg_class   c ON c.oid = t.tgrelid
WHERE  NOT t.tgisinternal
GROUP BY c.relname
ORDER BY trigger_count DESC;
```

Common Mistakes

1. Returning wrong value in BEFORE trigger

```
-- WRONG: BEFORE INSERT trigger returns NULL – silently swallows all inserts!
CREATE OR REPLACE FUNCTION bad_trigger() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
    -- forgot to RETURN NEW
```

```

        RETURN NULL; -- Every INSERT into this table is silently discarded
    END;
    $$;

-- CORRECT
CREATE OR REPLACE FUNCTION good_trigger() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
    NEW.created_at := now();
    RETURN NEW; -- Always return NEW for BEFORE INSERT/UPDATE
END;
    $$;

```

2. Forgetting FOR EACH ROW for per-row logic

```

-- WRONG: FOR EACH STATEMENT – OLD and NEW are NULL here
CREATE TRIGGER wrong_level
AFTER UPDATE ON orders
FOR EACH STATEMENT -- OLD.status is NULL here!
EXECUTE FUNCTION trg_that_uses_OLD_and_NEW();

-- CORRECT
CREATE TRIGGER right_level
AFTER UPDATE ON orders
FOR EACH ROW -- OLD and NEW have values
EXECUTE FUNCTION trg_that_uses_OLD_and_NEW();

```

3. Not handling all TG_OP cases

```

-- WRONG: Only handles INSERT but trigger is on INSERT OR UPDATE
CREATE OR REPLACE FUNCTION bad_multi_op() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO log VALUES (NEW.id, 'inserted'); -- crashes on DELETE (NEW is NULL)
    RETURN NEW;
END;
    $$;

-- CORRECT
CREATE OR REPLACE FUNCTION good_multi_op() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO log VALUES (NEW.id, 'inserted');
        RETURN NEW;
    ELSIF TG_OP = 'UPDATE' THEN
        INSERT INTO log VALUES (NEW.id, 'updated');
        RETURN NEW;
    ELSIF TG_OP = 'DELETE' THEN
        INSERT INTO log VALUES (OLD.id, 'deleted');
        RETURN OLD;
    END IF;

```

```
END;  
$$;
```

4. Modifying NEW in an AFTER trigger

```
-- WRONG: Modifying NEW in AFTER trigger has no effect  
CREATE OR REPLACE FUNCTION useless_after() RETURNS trigger LANGUAGE plpgsql AS $$  
BEGIN  
    NEW.processed := TRUE; -- AFTER trigger: this change is NOT written  
    RETURN NEW;  
END;  
$$;  
-- CORRECT: Move data modification logic to a BEFORE trigger
```

5. Trigger on partitioned tables — applies to parent only

```
-- Triggers defined on a partitioned table (parent) do NOT automatically  
-- propagate to partitions in PostgreSQL < 13.  
-- In PostgreSQL 13+, you can create triggers on the parent and they  
-- fire for partition inserts, but the trigger function receives the  
-- partition table name, not the parent.  
  
-- Safest: define triggers on each partition individually (or use pg_partman).
```

Best Practices

1. **Keep trigger functions short** — do one thing. Chain multiple concerns through separate triggers (use naming order to control sequence).
2. **Use WHEN clauses** to avoid entering the function for every row when only a subset of updates are relevant.
3. **Name triggers descriptively** with a prefix that encodes order: `10_normalize_`, `20_validate_`, `90_audit_`.
4. **Use SECURITY DEFINER for audit triggers** so the function runs as its owner (who has INSERT on the audit table) regardless of which role fires the trigger.
5. **Avoid long-running work in triggers** — triggers execute synchronously within the transaction. Use `pg_notify` + a background worker for async side effects.
6. **Always test TRUNCATE separately** — TRUNCATE fires statement-level triggers. Per-row triggers do NOT fire on TRUNCATE.
7. **Document all triggers** using `COMMENT ON TRIGGER :`

```
COMMENT ON TRIGGER audit_orders ON orders IS  
    'Logs all DML changes to audit_log. Uses SECURITY DEFINER to bypass row-level
```

```
security on audit_log.';
```

8. **Do not use triggers for referential integrity** that a foreign key constraint can enforce — FKs are faster, clearer, and transactionally correct.
 9. **Test trigger disable/enable during bulk loads** in staging to measure the performance impact before doing it in production.
 10. **Prefer AFTER triggers for side effects** (notifications, summary updates) so that if the side effect fails, the original operation rolls back too.
-

Interview Questions & Answers

Q1. What is the difference between a BEFORE and AFTER trigger? When would you use each?

A BEFORE trigger fires before the row is written to the table. It can modify `NEW` (altering what gets stored) or return `NULL` to cancel the operation. Use it for data normalization, validation, and computed columns.

An AFTER trigger fires after the row is written. It cannot modify the written row but can see the committed data and make changes to other tables. Use it for audit logging, notifications, and maintaining denormalized aggregates.

Q2. What does RETURN NULL do in a BEFORE ROW trigger?

It suppresses the operation for that specific row. The INSERT, UPDATE, or DELETE does not happen for that row. No error is raised — it is a silent skip. This is useful for idempotency guards (deduplication) but dangerous if used accidentally.

Q3. What is an INSTEAD OF trigger and when is it required?

An INSTEAD OF trigger replaces the default DML action on a view. It is required when you want a multi-table view to be updatable. The trigger function receives `NEW` and `OLD` and is responsible for translating the view-level operation into the underlying table operations.

Q4. What is the difference between FOR EACH ROW and FOR EACH STATEMENT?

FOR EACH ROW fires once per affected row and has access to OLD/NEW records. FOR EACH STATEMENT fires once per SQL statement regardless of affected rows, has no OLD/NEW (use transition tables instead), and has lower overhead for bulk operations.

Q5. Why are triggers preferred over rules for DML?

Rules operate at the query-rewrite level and are notoriously hard to debug, can interact unexpectedly with views, and do not respect row-level security. Triggers are explicit, transactional, debuggable with RAISE NOTICE, and supported by standard SQL semantics. The PostgreSQL documentation itself recommends triggers over rules for DML.

Q6. How do you prevent an infinite trigger loop?

Use a session-local variable as a guard flag: `PERFORM set_config('app.in_trigger_x', 'true', true)` (the third argument `true` = local to current transaction). At the start of the trigger function, check this flag and return early if it is already set. Alternatively, design the system so triggers only fire on meaningful changes (using WHEN clauses or checking `NEW.col IS DISTINCT FROM OLD.col`).

Q7. A BEFORE INSERT trigger is defined but INSERTs seem to disappear silently. What is the likely cause?

The trigger function is returning `NULL` instead of `NEW`. In a BEFORE ROW trigger, returning `NULL` silently suppresses the INSERT for that row. Check the trigger function's RETURN statement.

Q8. How do you implement optimistic locking with triggers?

Add a `version BIGINT` column. A BEFORE UPDATE trigger checks that `NEW.version = OLD.version` (the application should send the current version). If they match, increment: `NEW.version := OLD.version + 1`. If they do not match, raise a `serialization_failure` exception. The application catches 0 rows updated and retries.

Q9. Can you create a trigger on a partitioned table?

Yes, from PostgreSQL 13 onward, a trigger on the parent partitioned table fires for all partitions. In earlier versions, triggers must be defined on each partition individually. Note that BEFORE ROW triggers on partition parents are not supported in all versions — always test the exact behavior on your PostgreSQL version.

Q10. What happens to triggers during TRUNCATE?

TRUNCATE fires statement-level triggers (FOR EACH STATEMENT) if defined with TRUNCATE in the event list. It does NOT fire per-row triggers. If you need per-row audit on TRUNCATE, you must use a statement-level trigger with transition tables or prevent TRUNCATE with a rule.

Exercises and Solutions

Exercise 1

Create a trigger on a `products` table that automatically sets `updated_at = now()` on every UPDATE, but only if the `price`, `name`, or `description` columns actually changed.

Solution:

```
CREATE OR REPLACE FUNCTION trg_products_updated_at()
RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
    NEW.updated_at := now();
    RETURN NEW;
END;
$$;

CREATE TRIGGER products_updated_at
BEFORE UPDATE OF price, name, description ON products
FOR EACH ROW
WHEN (
    OLD.price      IS DISTINCT FROM NEW.price OR
    OLD.name       IS DISTINCT FROM NEW.name OR
    OLD.description IS DISTINCT FROM NEW.description
)
EXECUTE FUNCTION trg_products_updated_at();
```

Exercise 2

Implement a soft-delete audit: when a row in `employees` has its `deleted_at` set from NULL to a timestamp, insert a record into `employee_deletions(employee_id, deleted_by, deleted_at)`.

Solution:

```
CREATE OR REPLACE FUNCTION trg_record_employee_deletion()
RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
    IF OLD.deleted_at IS NULL AND NEW.deleted_at IS NOT NULL THEN
        INSERT INTO employee_deletions (employee_id, deleted_by, deleted_at)
        VALUES (NEW.id, current_user, NEW.deleted_at);
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER record_employee_deletion
AFTER UPDATE OF deleted_at ON employees
FOR EACH ROW
WHEN (OLD.deleted_at IS NULL AND NEW.deleted_at IS NOT NULL)
EXECUTE FUNCTION trg_record_employee_deletion();
```

Exercise 3

Write a query to list all triggers in the current database, their tables, timing (BEFORE/AFTER), event, and whether they are currently enabled.

Solution:

```
SELECT
    n.nspname                                AS schema,
    c.relname                                AS table_name,
    t.tgname                                  AS trigger_name,
    CASE WHEN t.tgtype & 64 = 64 THEN 'INSTEAD OF'
        WHEN t.tgtype & 2 = 2 THEN 'BEFORE'
        ELSE 'AFTER' END                     AS timing,
    string_agg(
        CASE WHEN t.tgtype & 4 = 4 THEN 'INSERT'
            WHEN t.tgtype & 8 = 8 THEN 'DELETE'
            WHEN t.tgtype & 16 = 16 THEN 'UPDATE'
            WHEN t.tgtype & 32 = 32 THEN 'TRUNCATE' END,
        ' OR ' )                              AS events,
    CASE t.tgenabled
        WHEN 'O' THEN 'enabled'
        WHEN 'D' THEN 'disabled'
        WHEN 'R' THEN 'enabled (replica)'
        WHEN 'A' THEN 'enabled (always)'
    END                                        AS status,
    p.proname                                AS function_name
FROM    pg_trigger t
JOIN    pg_class   c ON c.oid = t.tgrelid
JOIN    pg_namespace n ON n.oid = c.relnamespace
JOIN    pg_proc    p ON p.oid = t.tgfoid
WHERE   NOT t.tgisinternal
```



```
AND n.nspname NOT IN ('pg_catalog', 'information_schema')
GROUP BY n.nspname, c.relname, t.tgname, t.tgtype, t.tgenabled, p.proname
ORDER BY schema, table_name, timing, trigger_name;
```

Cross-References

- **09_stored_procedures_functions.md** — PL/pgSQL fundamentals used in trigger functions
- **06_logical_decoding.md** — Alternative to triggers for change capture (no overhead on write path)
- **04_materialized_views.md** — INSTEAD OF triggers make views updatable, similar to materialized view refresh patterns
- **08_listen_notify.md** — Using pg_notify inside AFTER triggers for async event processing
- **../09_Transactions_Concurrency/** — Understanding transaction boundaries and when trigger side effects commit
- **../12_Production_PostgreSQL/** — Monitoring trigger overhead with pg_stat_user_functions